Line Follower Digital Twin

using Innexis Virtual System Interconnect (VSI)

Alaa Elshaer

March 14, 2026

Contents

1 Introduction **2**

2 Prerequisites **2**

3 Generating the Digital Twin **2**
3.1 Workspace Setup 2
3.2 Build Command File 2
3.3 Building the Digital Twin 3

4 Simulator Implementation **3**
4.1 Robot Kinematic Model 3

5 Controller Design **3**

6 Visualizer and KPI Collection **4**

7 Experiments **4**
7.1 Experiment 1 – PID Gain Tuning 4
7.2 Experiment 2 – Curved Path Tracking 4
7.3 Experiment 3 – Noise Rejection 4
7.4 Experiment 4 – PD vs PID 4

8 Building and Running the Simulation **4**

9 Results **4**

10 Conclusion **5**

1 Introduction

This project implements a **Digital Twin of a line-following robot** using the Innexis Virtual System Interconnect (VSI) framework.

The architecture consists of three main distributed components:

- **Simulator** – models the robot dynamics and environment
- **Controller** – implements the control algorithm
- **Visualizer** – displays simulation results and collects performance metrics

The components communicate through a **CAN-based digital backplane** automatically generated by VSI.

The goal is to evaluate controller performance under multiple experimental conditions such as controller tuning, curved path tracking, disturbance rejection, and controller comparison.

2 Prerequisites

Software Environment

- Innexis VSI 2025.x
- Python 3
- Linux environment
- Python libraries:
  - numpy
  - pyqtgraph
  - PyQt6

3 Generating the Digital Twin

The Digital Twin architecture is generated using the `vsiBuild` tool.

3.1 Workspace Setup

```
mkdir lineFollowerCapstone
cd lineFollowerCapstone
```

3.2 Build Command File

The configuration file `vsiBuildCommands` defines:

- system components
- communication ports
- CAN frames

Three Python components are used:

- simulator
- controller
- visualizer

Signals exchanged include:

- lateral_error
- heading_error
- x_pos
- y_pos
- vel_cmd

Frames are mapped to CAN IDs 10–14.

3.3 Building the Digital Twin

```
vsiBuild -f vsiBuildCommands
```

This generates the project directory:

```
lineFollowerDemo/
```

Structure:

- src/simulator
- src/controller
- src/visualizer
- src/server

4 Simulator Implementation

The simulator models a mobile robot following a reference trajectory.

4.1 Robot Kinematic Model

The robot follows a unicycle model:

$$\dot{x} = v\cos(\theta) \tag{1}$$
$$\dot{y} = v\sin(\theta) \tag{2}$$
$$\dot{\theta} = \omega \tag{3}$$

where:

- v – forward velocity
- ω – steering angular velocity

The simulator calculates:

- lateral path error
- heading error relative to path tangent

To emulate realistic sensors, Gaussian noise is added to measurements.

5 Controller Design

The steering command is generated using a PID controller:

$$\omega = K_p e + K_i \int e\,dt + K_d \frac{de}{dt} \tag{4}$$

Two independent PID controllers are implemented:

- lateral error controller
- heading error controller

The final steering command is:

$$\omega = PID_{lat} + PID_{head} \tag{5}$$

The system operates with different update frequencies:

- Controller: 100 Hz
- Simulator: 1000 Hz

6 Visualizer and KPI Collection

The visualizer displays simulation results using the provided `RealTimePlotter` module.

Displayed plots include:

- Robot trajectory vs reference path
- Lateral error vs time
- Heading error vs time
- Angular velocity command vs time

The visualizer also computes Key Performance Indicators (KPIs):

- Maximum lateral error
- Settling time
- Steady-state error

Results are exported to:

```
kpi_results.csv
```

7 Experiments

Four experiments were conducted.

7.1 Experiment 1 – PID Gain Tuning

Different controller gains were evaluated to determine stable performance.

Results stored in:

```
kpi_E1.csv
```

7.2 Experiment 2 – Curved Path Tracking

The robot followed a sinusoidal reference path to evaluate performance on curved trajectories.

7.3 Experiment 3 – Noise Rejection

Gaussian noise was injected into sensor measurements to test controller robustness.

7.4 Experiment 4 – PD vs PID

The integral term was disabled to compare PD and full PID performance.

8 Building and Running the Simulation

```
cd lineFollowerDemo
make build
make sim
```

After launching the simulation interface, run:

```
run
```

The simulation executes until the configured simulation time expires.

9 Results

Observed behavior:

- Lateral error gradually decreases
- Heading error converges to zero
- Controller stabilizes the robot trajectory

This demonstrates effective path tracking performance.

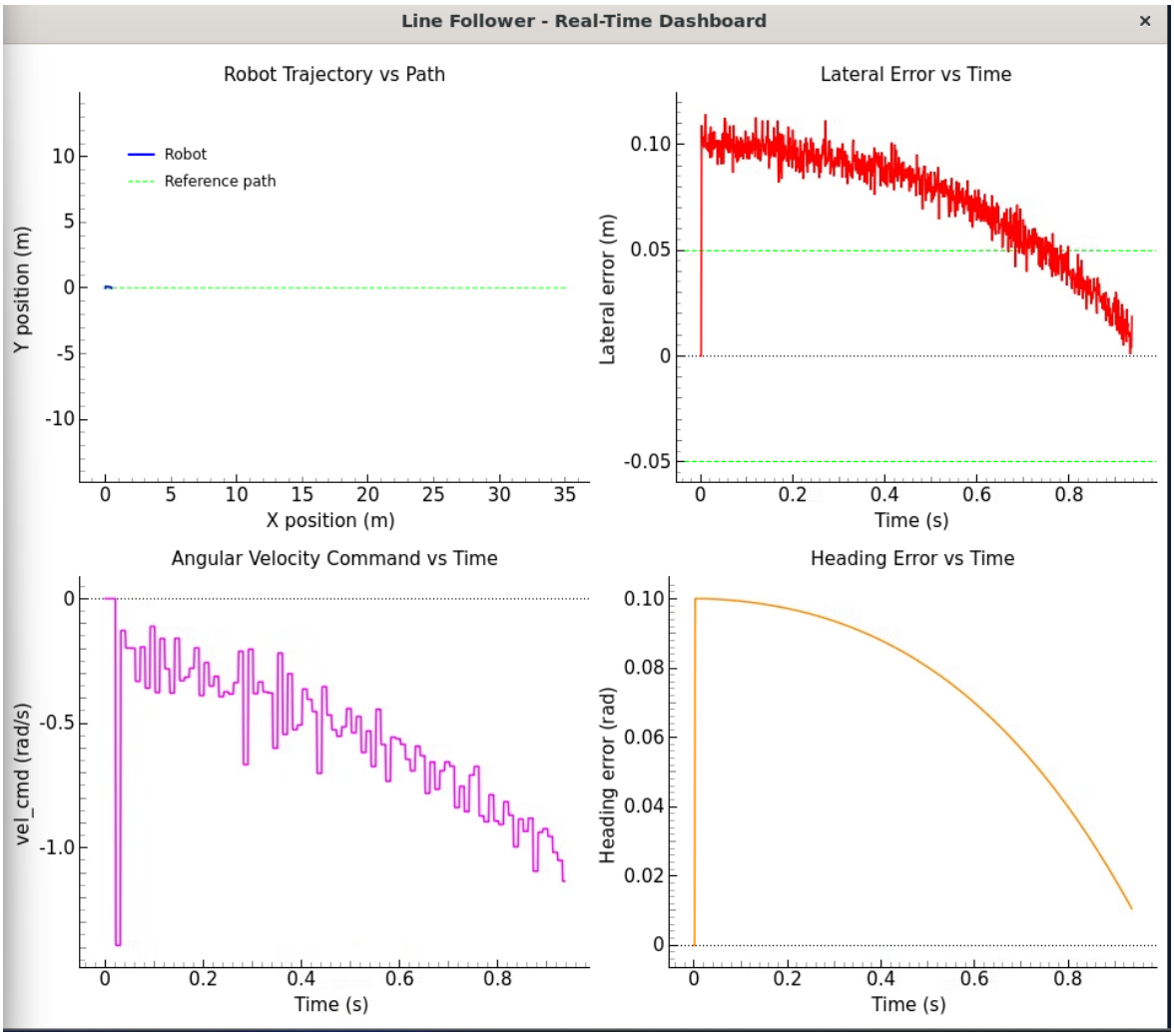

Figure 1: Real-time dashboard showing trajectory tracking and error signals

10 Conclusion

This project demonstrates a distributed robotic simulation architecture using the VSI digital twin framework.

The system integrates:

- robot dynamics simulation
- PID-based control
- real-time visualization

Experimental results show stable trajectory tracking across different operating conditions.

Future improvements may include:

- adaptive PID tuning
- advanced path planning
- machine-learning-based control